

Apptainer & Singularity for HPC

Lecture 1 of 4

*From Docker to Singularity:
Design, Security Model, and First Commands*

DATA SCIENCE AND ARTIFICIAL INTELLIGENCE – University of Trieste
Academic Year 2025/2026
Target platform: Leonardo @ CINECA

```
$ singularity --version
singularity version 3.11.4

$ singularity pull \
  docker://ncbi/blast:latest
INFO: Converting OCI blobs...
INFO: Starting build...
INFO: Creating SIF file...

$ ls -lh blast_latest.sif
-rwxr-xr-x 1 alice alice
634M blast_latest.sif

$ singularity exec \
  blast_latest.sif \
  blastn -version
blastn: 2.15.0+
Package: blast 2.15.0, ...
```

Module Learning Objectives

Four lectures + four practical exercises on Leonardo

Explain

Why Docker cannot run on shared HPC clusters; the design principles behind Apptainer

Build

Custom SIF images from definition files and from existing Docker images

Configure

OpenMP thread affinity and NUMA-aware binding inside containers on Sapphire Rapids

Deploy

Hybrid MPI+OpenMP parallel applications across multiple nodes using host MPI + singularity exec

Run

GPU-accelerated workloads on NVIDIA A100 using the --nv passthrough mechanism

Evaluate

Trade-offs: when containers add value vs. native module stacks

Why Containers on HPC?

The four pain points that module systems cannot solve

1

Dependency conflicts

Library version A needed by application X conflicts with version B required by Y. Both must coexist.

2

Reproducibility

Reproduce the exact software stack used 18 months ago for a Nature paper — with identical binary versions.

3

Portability

Workflow developed on a MacBook must run unchanged on Leonardo without re-installing anything.

4

Docker Hub access

A tool exists only as a Docker image. Leonardo has no Docker. Apptainer bridges the gap.

Key insight: containers do NOT virtualise the kernel: the host kernel is shared. Startup overhead is near zero; InfiniBand and GPU access are direct.

Why Docker Cannot Run on Shared HPC Clusters

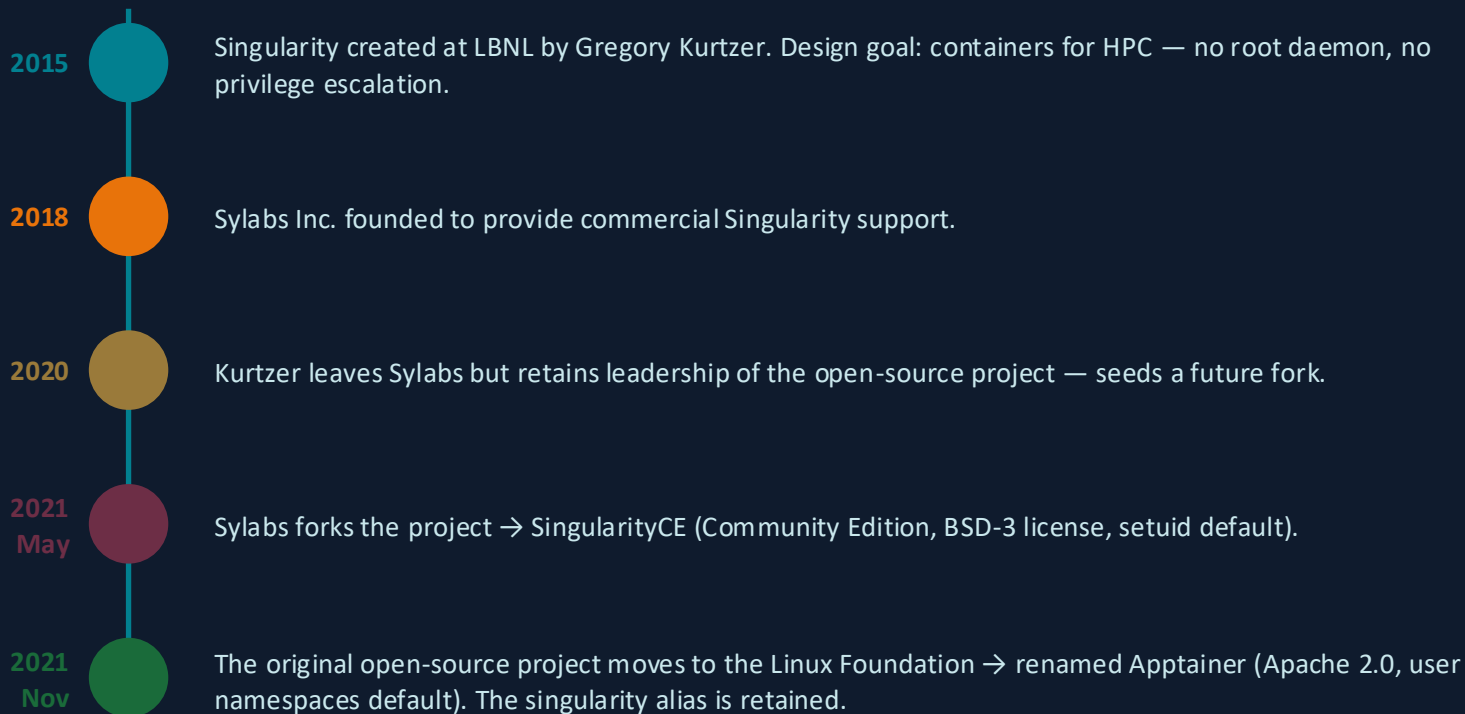
Each property is a security or performance failure mode

Docker Property	Impact on HPC	Severity
Persistent root daemon (dockerd)	Security risk on shared systems; unavailable on login nodes	Critical
Container can escalate to root on host	A user could gain root on the login node	Critical
Isolated bridge network (default)	Breaks InfiniBand, MPI collective operations	Critical
Overlayfs via daemon	Incompatible with Lustre and GPFS parallel filesystems	High
Layered OCI format (thousands of small files)	Severe metadata load on Lustre: poor I/O performance	High
Runs as root inside container by default	Violates all HPC site security policies	High

Fundamental problem: Docker assumes a trusted admin configuring containers for untrusted users. HPC clusters must support untrusted users running their own containers.

A Brief History: Singularity → Apptainer

Understanding the naming confusion you will encounter in the wild



Three Products, One Compatible Format

All derive from Singularity 2015; SIF images are fully interchangeable

Singularity	SingularityCE	Apptainer
Sylabs (commercial) License: Proprietary Command: <code>singularity</code> Enterprise support cloud.sylabs.io	Sylabs (community) License: BSD-3 Command: <code>singularity</code> Uses setuid binary by default	Linux Foundation License: Apache 2.0 Command: <code>apptainer</code> (+ <code>singularity</code> alias) User namespaces by default

CINECA (Leonardo, 2025): The installed runtime is called Singularity in documentation. The singularity command is available on login nodes and compute nodes. Apptainer provides a full singularity alias: all commands and SINGULARITY_ environment variables work identically.

Apptainer's Security Model: You Stay You

The container process runs as you, not as root....always!

HOST (Leonardo compute node)

User: alice (uid=1001)

```
singularity exec my_container.sif /bin/bash
```

Files created inside the container are owned by alice: no permission conflicts

CONTAINER user space

User: alice (uid=1001) ← SAME AS HOST

Cannot sudo, cannot escalate privileges

/home/alice auto-mounted (Lustre home visible)

Host kernel shared - no virtualisation overhead

sudo inside the container fails: privilege escalation blocked

Lustre scratch (\$SCRATCH) accessible at the same path as on the host

Apptainer vs Docker: Feature Comparison

Same concept, radically different security model

Feature	Apptainer / Singularity	Docker
Root daemon required	No	Yes (dockerd)
User identity inside container	Same as host user	root (by default)
Image format	Single SIF file (SquashFS)	Layered OCI (directory tree)
Parallel filesystem (Lustre/GPFS)	Compatible	Incompatible
MPI / InfiniBand	Full access via host stack	Requires --network=host
NVIDIA GPU	--nv or --nvccli flag	--gpus + nvidia-container-toolkit daemon
Privilege escalation	Blocked	Possible
Build requires root?	Yes (build only on laptop)	Yes (daemon running as root)
Pull from Docker Hub	Yes (docker:// prefix)	Yes
Suitable for HPC login nodes	Yes	No

The SIF Image Format: One Large File

SquashFS compression + cryptographic signatures + optional encryption

blast_latest.sif

SquashFS filesystem

Complete container user space (compressed read-only)

Optional GPG signature

Cryptographic authenticity verification

Optional AES-256

Encryption for proprietary or sensitive images

Embedded metadata

Labels, definition file, build timestamp

Why SIF is right for Lustre

- One large compressed file performs orders of magnitude better than tens of thousands of small Docker layer files on Lustre
- Lustre/GPFS are optimised for large sequential I/O: exactly what SIF provides
- Docker's layered format expands to 50,000+ files in `~/.docker/` causes severe metadata load
- SIF is immutable by default: no accidental modification on shared filesystem

On Leonardo: store SIF files in `$SCRATCH`, set `SINGULARITY_CACHEDIR=$SCRATCH/.singularity_cache`. Never store SIF or cache in `$HOME`. You will exhaust your home quota.

Essential Apptainer / Singularity Commands

These are the six verbs you will use in every session

```
singularity pull docker://ncbi/blast:latest
```

Download + convert Docker image from Hub → SIF file

```
singularity shell my.sif
```

Interactive shell inside the container

```
singularity exec my.sif CMD [ARGS]
```

Execute single command inside container: main production pattern

```
singularity run my.sif
```

Execute the %runscript defined at build time

```
singularity build my.sif my.def
```

Build SIF from definition file (requires sudo on laptop)

```
singularity inspect --labels my.sif
```

Show metadata, labels, embedded definition file

Automatic Bind Mounts on Leonardo

Host filesystems visible inside the container: no --bind needed for standard paths

Always auto-mounted (Apptainer default)

```
$HOME      → /home/<username>
$PWD       → <same path>
/tmp       → /tmp
/proc, /sys, /dev

# Your Lustre scratch is $PWD-relative:
# cd $SCRATCH → /leonardo/scratch/...
# visible inside container automatically
```

CINECA adds on Leonardo

```
/leonardo/home/ → /leonardo/home/
/leonardo/scratch/ → /leonardo/scratch/
/leonardo/work/ → /leonardo/work/

# Add extra mounts:
singularity exec \
    --bind /data:/data my.sif cmd
# or: export SINGULARITY_BIND=/data
```

Practical consequence

- A Python script in \$SCRATCH runs inside the container reading and writing to \$SCRATCH: no data copy needed
- Output files written by the container appear immediately on the host Lustre filesystem
- SLURM environment variables (SLURM_CPUS_PER_TASK, SLURM_NODEID, etc.) are inherited automatically

SLURM + Singularity: The Production Pattern

Never pull inside a job; always use singularity exec in the job script

```
#!/bin/bash
#SBATCH --job-name=blast_container
#SBATCH --nodes=1 --ntasks-per-node=1 --cpus-per-task=8
#SBATCH --mem=16G --time=00:15:00
#SBATCH --partition=dcgp_usr_prod --account=<your_account>
#SBATCH --output=%x.%j.out

# Redirect cache to scratch (never $HOME on Leonardo)
export SINGULARITY_CACHEDIR=$SCRATCH/.singularity_cache

# Use pre-pulled SIF – never pull inside a job
SIF=$SCRATCH/containers/blast_latest.sif

# SLURM_CPUS_PER_TASK is visible inside the container
singularity exec $SIF blastn \
    -query $SCRATCH/query.fasta \
    -db $SCRATCH/blastdb \
    -num_threads ${SLURM_CPUS_PER_TASK} \
    -outfmt 6 -out $SCRATCH/results.txt
```

Three rules: (1) Pull images before the job, not inside it. (2) SIF files go in \$SCRATCH, not \$HOME. (3) Use singularity exec, not singularity run, for explicit command control.

Exercise 1 — First Contact on Leonardo

Scientific context: running NCBI BLAST via container (not available as a CINECA module)

Part 1 — Pull and Inspect [8 pts]

- Pull ncbi/blast:latest from Docker Hub → SIF on \$SCRATCH
- singularity inspect --labels, --deffile
- Measure SIF file size; compare with Docker Hub size
- Identify base OS: `exec blast.sif cat /etc/os-release`

Part 2 — Filesystem & Permissions [10 pts]

- Verify \$HOME and \$SCRATCH visible inside container
- Create FASTA file → `makeblastdb` → `blastn` pipeline
- Try `sudo` inside the container — observe failure
- Compare `id` inside vs outside container

Part 3 — SLURM Batch Job [7 pts]

- Write a batch script using pre-pulled SIF
- Use `${SLURM_CPUS_PER_TASK}` for `-num_threads`
- Explain: why never singularity pull inside a job?
- Explain: why SIF in \$SCRATCH not \$HOME?

Exercise 1: Setup Commands on Leonardo

Run these in sequence on your terminal after ssh into Leonardo

```
# 1. Login to Leonardo
ssh <username>@login.leonardo.cineca.it

# 2. CRITICAL: redirect cache away from $HOME
export SINGULARITY_CACHEDIR=$SCRATCH/.singularity_cache
mkdir -p $SINGULARITY_CACHEDIR

# 3. Verify runtime version
singularity --version
# singularity version 3.11.4

# 4. Request interactive allocation on DCGP (CPU-only, fast queue)
salloc --nodes=1 --ntasks-per-node=1 --cpus-per-task=4 --mem=8G \
    --time=01:00:00 --partition=dcgp_usr_prod \
    --account=<your_account>

# 5. Pull BLAST (conversion from OCI → SIF happens automatically)
singularity pull $SCRATCH/containers/blast_latest.sif \
    docker://ncbi/blast:latest
```

Understanding Check: Six Questions to Answer

Reflect on these before moving to Lecture 2

Q1

Why does singularity exec blast.sif sudo ls / fail even though you are alice on the host?

Q2

You write a file to \$SCRATCH from inside the container. Why does it appear on the host without any special configuration?

Q3

SLURM_CPUS_PER_TASK is set on the host by SLURM. How does the container 'see' this variable?

Q4

Why should singularity pull never be called from inside a SLURM job script on a compute node?

Q5

A colleague builds a SIF on their Ubuntu 24.04 laptop and sends it to you. You run it on Leonardo (RHEL 8 kernel). What executes — the Ubuntu user space or the RHEL kernel?

Q6

A Docker image on Docker Hub is reported as 1.8 GB. After singularity pull, the SIF is 1.2 GB. Where did the 600 MB go?

Common Mistakes to Avoid

Problems you will encounter....and how to diagnose them

Cache in \$HOME

SINGULARITY_CACHEDIR defaults to ~/.singularity — this fills your home quota quickly on Leonardo. Always redirect to \$SCRATCH.

Pulling inside a job

Compute nodes on Leonardo may not have internet access. Pull images on the login node before submitting the job.

Sandbox on Lustre

Sandbox images (directories) create tens of thousands of small files — catastrophic for Lustre metadata. Build sandboxes on your laptop only.

Wrong architecture SIF

An image built for arm64 (Apple Silicon) will not run on Leonardo's x86_64 nodes. Always build for x86_64 or use --platform linux/amd64 in Docker.

Using :latest tags

docker://ubuntu:latest points to a different image next month. Use docker://ubuntu:22.04 for reproducibility. The SIF captures the exact binary — version it in the filename: ubuntu_22.04.sif

Container vs Module: When to Use Which

Containers add overhead: use them when they provide clear value

Situation	Recommendation	Reason
Standard HPC codes (GROMACS, LAMMPS, WRF): CINECA provides modules	module load	CINECA modules are optimised for Leonardo hardware (AVX-512, UCX)
Custom Python/R environment with specific library versions	Container	pip/conda environments inside containers are isolated and portable
Reproducing a published result from 2 years ago	Container	Freeze exact environment; modules may have been updated or removed
Workflow originally on Docker Hub (no source)	Container	singularity pull converts Docker image automatically
Maximum MPI scaling on InfiniBand (100+ nodes)	module load	Host MPI+UCX is tuned for Leonardo's HDR InfiniBand network
Interactive data analysis (Jupyter, R Studio)	Container	Custom kernels; version isolation; no root needed
Proprietary or pre-compiled binary only available as Docker image	Container	Only viable option on a cluster without Docker

Leonardo @ CINECA: System Quick Reference

Relevant hardware details for the exercises in this module

DCGP Partition (CPU-only)

1536 compute nodes

2 × Intel Sapphire Rapids (56 cores each)

112 logical CPUs/node (hyperthreading)

~512 GB DDR5 DRAM per node

2 NUMA nodes (sockets) per node

InfiniBand HDR 200 Gb/s (Dragonfly+)

Partition: dcgp_usr_prod

No GP, pure CPU workloads

Booster Partition (GPU)

3456 compute nodes

1 × Intel Ice Lake (32 cores)

4 × NVIDIA A100 SXM4 64GB per node

~512 GB DRAM per node

InfiniBand HDR 200 Gb/s (Dragonfly+)

NVLink between GPUs on same node

Partition: boost_usr_prod

Singularity: --nv or --nvccli for GPUs

Singularity command available on login nodes and all compute nodes. No module load needed.

Key Takeaways

Five things to remember before Lecture 2

1

Docker ≠ HPC containers

Root daemon, privilege escalation, isolated network, OCI format: all incompatible with shared HPC systems. Apptainer was designed from scratch to solve these problems.

2

SIF: one large file

SquashFS-compressed single file performs orders of magnitude better than Docker's layered OCI format on Lustre. Store in `$SCRATCH`, never `$HOME`.

3

You stay you

The container process runs as the same user as the host. Files it creates are yours. Privilege escalation is blocked.

4

Auto-mounts, no data copy

`$HOME`, `$PWD`, `$SCRATCH` (via Lustre path), and SLURM variables are all visible inside the container automatically.

5

Apptainer IS Singularity

Three products (Singularity, SingularityCE, Apptainer) share SIF format. On Leonardo, use the singularity command regardless of which runtime is installed.

What's Next

L2: Building Your First Image

Definition files in detail (%post, %environment, %test), build workflow on laptop → scp → Leonardo, converting Docker archives, NVIDIA NGC, sandbox mode, reproducibility best practices.

L3: OpenMP: Affinity and NUMA

OMP_PROC_BIND, OMP_PLACES, SLURM --cpu-bind, numactl inside containers. DGEMM scaling study on DCGP (2×56 core Sapphire Rapids).

L4: MPI and GPU in Containers

Host MPI + singularity exec pattern, --nv GPU passthrough, CUDA driver injection, GROMACS on 4×A100 Booster nodes. SAXPY bandwidth benchmark.

Exam: 5-Day Practical Exam

Day 1: VM cluster + SLURM. Day 2: Container cluster. Day 3: Benchmarking. Day 4: AWS deployment. Day 5: Report + presentation.